DBMS Lab Sheet 3

Lab Sheet 3: SQL Functions - Number, Aggregate, Character, Conversion and Date functions

Run both setup scripts first: the first creates the tables and the second loads the sample data. Outputs shown are for that sample data. All functions are for MySQL 8.0.

Setup 1: create tables

```
USE CollegeDB;
SET SQL_SAFE_UPDATES = 0;   -- lets UPDATE/DELETE run without a key in WHERE (Workbench)

DROP TABLE IF EXISTS Student, Student_Details, Employee, Department,
                     Product, Customer, Course, Student_Demo;

CREATE TABLE Department (
    Dept_ID   INT PRIMARY KEY,
    Dept_Name VARCHAR(50) NOT NULL,
    Location  VARCHAR(50)
);
CREATE TABLE Course (
    Course_ID   INT PRIMARY KEY,
    Course_Name VARCHAR(50) NOT NULL,
    Duration    VARCHAR(20),
    Fees        DECIMAL(10,2)
);
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name       VARCHAR(50) NOT NULL,
    Age        INT,
    Course     VARCHAR(30),
    City       VARCHAR(30),
    Dept_ID    INT
);
CREATE TABLE Employee (
    Emp_ID     INT PRIMARY KEY,
    Emp_Name   VARCHAR(50) NOT NULL,
    Department VARCHAR(30),
    Salary     DECIMAL(10,2),
    Email      VARCHAR(80) UNIQUE,
    Join_Date  DATE,
    Manager_ID INT,
    Dept_ID    INT
);
CREATE TABLE Product (
    Product_ID   INT PRIMARY KEY,
    Product_Name VARCHAR(50) NOT NULL,
    Price        DECIMAL(10,2) CHECK (Price > 0),
    Quantity     INT
);
CREATE TABLE Customer (
    Customer_ID   INT PRIMARY KEY,
    Customer_Name VARCHAR(50) NOT NULL,
    Email         VARCHAR(80),
    Phone         VARCHAR(15)
);
```

Setup 2: load sample data

```
USE CollegeDB;
SET SQL_SAFE_UPDATES = 0;
TRUNCATE TABLE Student;   TRUNCATE TABLE Employee;   TRUNCATE TABLE Department;
TRUNCATE TABLE Course;    TRUNCATE TABLE Product;    TRUNCATE TABLE Customer;

INSERT INTO Department VALUES
    (1,'Computer Science','Block A'), (2,'Mathematics','Block B'),
    (3,'Physics','Block C'),          (4,'Commerce','Block D'),
    (5,'Chemistry','Block E');

INSERT INTO Course VALUES
    (101,'BCA','3 Years',90000), (102,'BBA','3 Years',85000),
    (103,'B.Sc','3 Years',60000), (104,'MCA','2 Years',110000),
    (105,'B.Com','3 Years',45000);

INSERT INTO Student VALUES
    (1,'Aarav Sharma',20,'BCA','Meerut',1),
    (2,'Priya Singh',21,'BBA','Delhi',4),
    (3,'Rohan Verma',19,'B.Sc','Meerut',3),
    (4,'Sneha Gupta',22,'MCA','Noida',1),
    (5,'Aman Khan',20,'BCA','Lucknow',1),
    (6,'Kavya Jain',21,'B.Sc','Delhi',2),
    (7,'Rahul Yadav',23,'MCA','Meerut',NULL),
    (8,'Neha Tyagi',20,'B.Com','Ghaziabad',4);

INSERT INTO Employee VALUES
    (1,'Anil Mehta','Computer Science',85000,'anil@college.edu','2015-06-15',NULL,1),
    (2,'Sunita Rao','Computer Science',60000,'sunita@college.edu','2018-07-01',1,1),
    (3,'Vikram Joshi','Mathematics',70000,'vikram@college.edu','2016-08-20',NULL,2),
    (4,'Pooja Malik','Mathematics',45000,'pooja@college.edu','2020-01-10',3,2),
    (5,'Manoj Tiwari','Commerce',52000,'manoj@college.edu','2019-03-25',NULL,4),
    (6,'Deepa Nair','Commerce',38000,'deepa@college.edu','2021-09-05',5,4),
    (7,'Imran Ali','Physics',65000,'imran@college.edu','2017-11-12',NULL,3),
    (8,'Ritu Saxon','Computer Science',47000,'ritu@college.edu','2022-02-14',1,1);

INSERT INTO Product VALUES
    (1,'Laptop',55000.00,10), (2,'Mouse',500.00,50), (3,'Keyboard',1200.00,0),
    (4,'Monitor',9500.00,15), (5,'USB Drive',650.50,0);

INSERT INTO Customer VALUES
    (1,'Ankit Arora','ankit@example.com','9876543210'),
    (2,'Meena Kapoor','meena@example.com','9876501234'),
    (3,'Sandeep Rana','sandeep@example.com','9812345678');
```

Q1. Demonstrate ABS() with suitable examples.

```
SELECT ABS(-25) AS Abs_Negative, ABS(25) AS Abs_Positive, ABS(-7.5) AS Abs_Decimal;

-- on table data: distance of each salary from 60000
SELECT Emp_Name, Salary, ABS(Salary - 60000) AS Diff_From_60K FROM Employee;
```

Output:

Abs_Negative	Abs_Positive	Abs_Decimal
25	25	7.5

Q2. Demonstrate ROUND() with suitable examples.

```
SELECT ROUND(123.4567, 2)  AS Two_Places,
       ROUND(123.4567, 0)  AS Whole_Number,
       ROUND(123.4567, -1) AS Nearest_Ten;
```

Output:

Two_Places	Whole_Number	Nearest_Ten
123.46	123	120

A negative second argument rounds to the left of the decimal point.

Q3. Demonstrate CEIL() and FLOOR().

```
SELECT CEIL(4.2)   AS Ceil_Pos,  FLOOR(4.8)   AS Floor_Pos,
       CEIL(-4.2)  AS Ceil_Neg,  FLOOR(-4.2)  AS Floor_Neg;
```

Output:

Ceil_Pos	Floor_Pos	Ceil_Neg	Floor_Neg
5	4	-4	-5

CEIL rounds towards +infinity and FLOOR towards -infinity, so they differ for negative numbers.

Q4. Demonstrate MOD() with suitable examples.

```
SELECT MOD(17, 5) AS Mod_Fn, 17 % 5 AS Percent_Op, 17 MOD 5 AS Mod_Op, MOD(-17, 5) AS Neg;

-- odd / even student ids
SELECT Student_ID, Name,
       IF(MOD(Student_ID, 2) = 0, 'Even', 'Odd') AS Id_Type
FROM Student;
```

Output:

Mod_Fn	Percent_Op	Mod_Op	Neg
2	2	2	-2

In MySQL the result of MOD takes the sign of the dividend, hence MOD(-17,5) = -2.

Q5. Calculate the rounded salary of employees.

```
SELECT Emp_Name,
       Salary,
       ROUND(Salary / 12)  AS Monthly_Salary,
       ROUND(Salary, -4)   AS Nearest_10K
FROM Employee;
```

Q6. Find the total number of students using COUNT().

```
SELECT COUNT(*) AS Total_Students FROM Student;

-- COUNT(column) ignores NULL values (Rahul Yadav has no department)
SELECT COUNT(Dept_ID) AS Students_With_Dept FROM Student;
```

Output:

Total_Students
8

Q7. Find the total salary of all employees using SUM().

```
SELECT SUM(Salary) AS Total_Salary FROM Employee;
```

Output:

Total_Salary
462000.00

Q8. Find the average salary using AVG().

```
SELECT AVG(Salary) AS Average_Salary FROM Employee;
```

Output:

Average_Salary
57750.000000

Q9. Find the highest salary using MAX().

```
SELECT MAX(Salary) AS Highest_Salary FROM Employee;

-- who earns it?
SELECT Emp_Name, Salary FROM Employee
WHERE Salary = (SELECT MAX(Salary) FROM Employee);
```

Output:

Highest_Salary
85000.00

Q10. Find the lowest salary using MIN().

```
SELECT MIN(Salary) AS Lowest_Salary FROM Employee;
```

Output:

Lowest_Salary
38000.00

Q11. Demonstrate UPPER() on student names.

```
SELECT Name, UPPER(Name) AS Upper_Name FROM Student
ORDER BY Student_ID LIMIT 3;
```

Output:

Name	Upper_Name
Aarav Sharma	AARAV SHARMA
Priya Singh	PRIYA SINGH
Rohan Verma	ROHAN VERMA

Q12. Demonstrate LOWER() on student names.

```
SELECT Name, LOWER(Name) AS Lower_Name FROM Student
ORDER BY Student_ID LIMIT 3;
```

Output:

Name	Lower_Name
Aarav Sharma	aarav sharma
Priya Singh	priya singh
Rohan Verma	rohan verma

Q13. Demonstrate LENGTH() on student names.

```
SELECT Name, LENGTH(Name) AS Name_Length FROM Student
ORDER BY Student_ID LIMIT 3;
```

Output:

Name	Name_Length
Aarav Sharma	12
Priya Singh	11
Rohan Verma	11

LENGTH counts bytes; use CHAR_LENGTH(Name) to count characters when names may contain non-English letters.

Q14. Demonstrate CONCAT() using first and last names.

```
SELECT Name,
       SUBSTRING_INDEX(Name, ' ', 1)  AS First_Name,
       SUBSTRING_INDEX(Name, ' ', -1) AS Last_Name,
       CONCAT(SUBSTRING_INDEX(Name, ' ', -1), ', ',
              SUBSTRING_INDEX(Name, ' ', 1)) AS Last_First
FROM Student ORDER BY Student_ID LIMIT 3;
```

Output:

Name	First_Name	Last_Name	Last_First
Aarav Sharma	Aarav	Sharma	Sharma, Aarav
Priya Singh	Priya	Singh	Singh, Priya
Rohan Verma	Rohan	Verma	Verma, Rohan

The table stores the full name in one column, so SUBSTRING_INDEX splits it into first and last name before CONCAT joins the parts again.

Q15. Demonstrate SUBSTRING()/SUBSTR() with suitable examples.

```
SELECT SUBSTRING('Database Management', 1, 8) AS First_8,
       SUBSTR('Database Management', 10)       AS From_10,
       SUBSTRING('Database', -4)               AS Last_4;

-- first three letters of every student name
SELECT Name, SUBSTRING(Name, 1, 3) AS Short_Name FROM Student;
```

Output:

First_8	From_10	Last_4
Database	Management	base

Q16. Convert a number into a character/string value.

```
SELECT Emp_Name,
       CAST(Salary AS CHAR)         AS Salary_Text,
       CONCAT('Rs. ', Salary)       AS Salary_Label,
       CONCAT('Rs. ', FORMAT(Salary, 0)) AS Salary_Formatted
FROM Employee;
```

CONCAT converts numbers to strings automatically; FORMAT adds thousands separators.

Q17. Convert a string into a numeric value.

```
SELECT '123.45' + 0              AS Implicit,
       CAST('250' AS UNSIGNED)   AS Explicit_Int,
       '10' * '5'                AS Product;
```

Output:

Implicit	Explicit_Int	Product
123.45	250	50

Q18. Convert a date value into a different format.

```
SELECT DATE_FORMAT('2024-08-15', '%d/%m/%Y')          AS Day_First,
       DATE_FORMAT('2024-08-15', '%W, %d %M %Y')      AS Long_Format,
       STR_TO_DATE('15-08-2024', '%d-%m-%Y')          AS Back_To_Date;
```

Output:

Day_First	Long_Format	Back_To_Date
15/08/2024	Thursday, 15 August 2024	2024-08-15

DATE_FORMAT turns a date into formatted text; STR_TO_DATE does the reverse.

Q19. Demonstrate CAST() with suitable examples.

```
SELECT CAST('99.567' AS DECIMAL(10,2)) AS To_Decimal,
       CAST('2026-10-13' AS DATE)      AS To_Date,
       CAST(2026 AS CHAR)              AS To_Char;

SELECT Name, CAST(Age AS CHAR) AS Age_Text FROM Student;
```

Output:

To_Decimal	To_Date	To_Char
99.57	2026-10-13	2026

Q20. Demonstrate CONVERT() where supported by the DBMS.

```
SELECT CONVERT('100', SIGNED)         AS To_Int,
       CONVERT('2026-10-13', DATE)    AS To_Date,
       CONVERT(Name USING utf8mb4)    AS Name_Utf8
FROM Student LIMIT 3;
```

MySQL has two forms: CONVERT(expr, type) like CAST, and CONVERT(expr USING charset) for character-set conversion.

Q21. Display the current date using a suitable date function.

```
SELECT CURDATE()      AS Today,
       CURRENT_DATE() AS Today_Alias,
       NOW()          AS Date_And_Time,
       CURTIME()      AS Current_Time;
```

Q22. Extract the year, month and day from a date.

```
SELECT Emp_Name,
       Join_Date,
       YEAR(Join_Date)      AS Join_Year,
       MONTH(Join_Date)     AS Join_Month,
       DAY(Join_Date)       AS Join_Day,
       MONTHNAME(Join_Date) AS Month_Name
FROM Employee;
```

Q23. Find the difference between two dates.

```
SELECT DATEDIFF('2026-12-31', '2026-10-13') AS Days_Between;

-- years of service of each employee
SELECT Emp_Name, Join_Date,
       TIMESTAMPDIFF(YEAR, Join_Date, CURDATE()) AS Years_Of_Service
FROM Employee;
```

Output:

Days_Between
79

Q24. Add a specified number of days to a date.

```
SELECT DATE_ADD('2026-10-13', INTERVAL 30 DAY)  AS Plus_30_Days,
       DATE_SUB('2026-10-13', INTERVAL 7 DAY)   AS Minus_7_Days,
       '2026-10-13' + INTERVAL 1 MONTH          AS Plus_1_Month;
```

Output:

Plus_30_Days	Minus_7_Days	Plus_1_Month
2026-11-12	2026-10-06	2026-11-13

Q25. Display employees' joining dates in a required date format.

```
SELECT Emp_Name,
       DATE_FORMAT(Join_Date, '%d-%b-%Y')  AS Joined_On,
       DATE_FORMAT(Join_Date, '%d/%m/%Y')  AS Joined_DMY
FROM Employee ORDER BY Join_Date;
```

Output:

Emp_Name	Joined_On	Joined_DMY
Anil Mehta	15-Jun-2015	15/06/2015
Vikram Joshi	20-Aug-2016	20/08/2016
Imran Ali	12-Nov-2017	12/11/2017
Sunita Rao	01-Jul-2018	01/07/2018

Only the first four of the eight rows are shown in the output.